Women In Data™ Toronto Chapter · Oct 21, 2026

The Machinery Changed

From OLAP cubes to agentic AI, and the discipline that stayed the same

Prashanth Soma

Senior Data & AI Platform Architect

What we are covering today

THE TALK

1. **Thesis**
   The machinery changed. The discipline didn't.
2. **Five eras of data architecture**
   AS/400 → cube → MPP → lakehouse → agent. What each solved, created, and left unchanged.
3. **The constant underneath**
   What never changed across 30 years — and why it matters now more than ever.

LIVE DEMOS — LOCAL MACHINE, NO CLOUD

1. **MCP tool docstrings**
   How a Python docstring becomes an agent's reasoning instruction.
2. **Ontology demo — Protégé + MCP**
   "Diabetes" → ICD-10 codes. How agents resolve meaning before querying.
3. **NL2SQL — live agent query**
   Plain English in. Validated SQL out. Powered by Ollama on this machine.

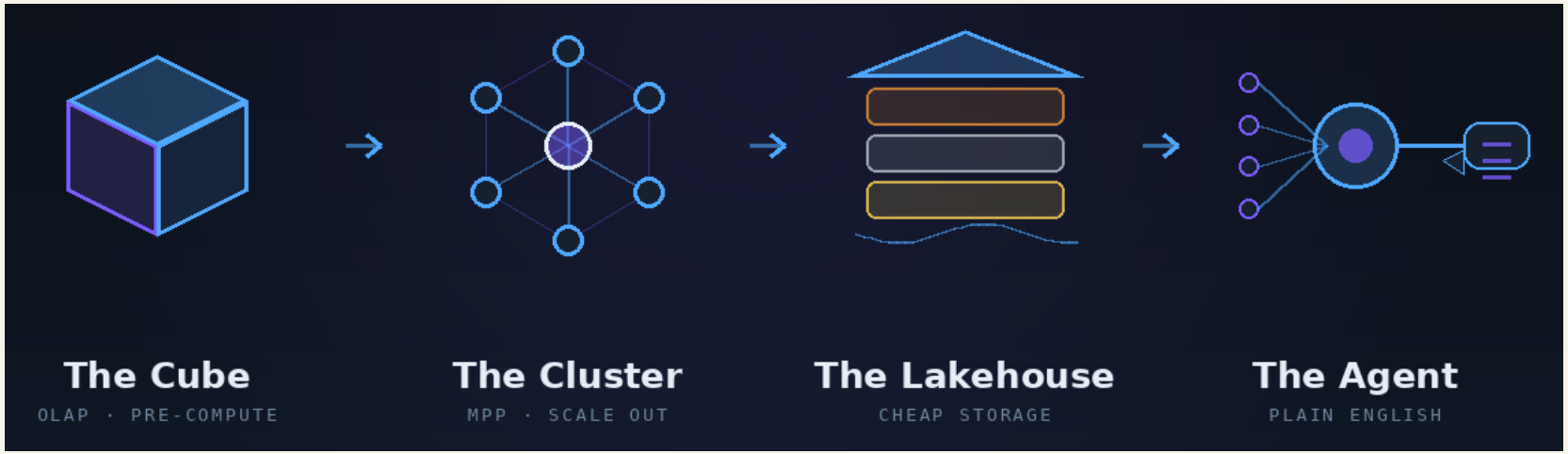
Everything running locally. No vendor dependency.

Q&A · References and image credits at the end

A number appears on a dashboard. How do you know it's right?

Hold that question. We will come back to it.

Four eras, one question



Era 1

OLAP cube

Pre-compute the answers

Era 2

MPP cluster

Spread the work across machines

Era 3

Lakehouse

One copy of data for BI and ML

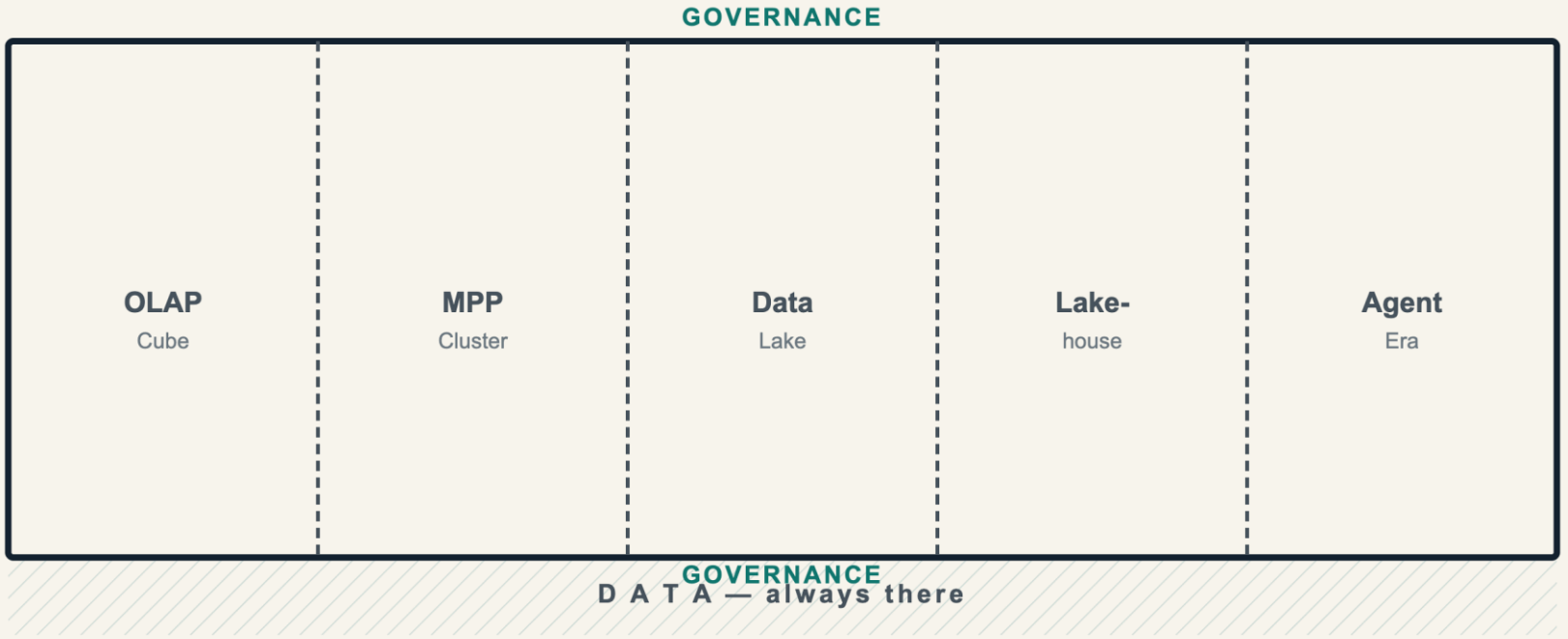
Era 4

Agent

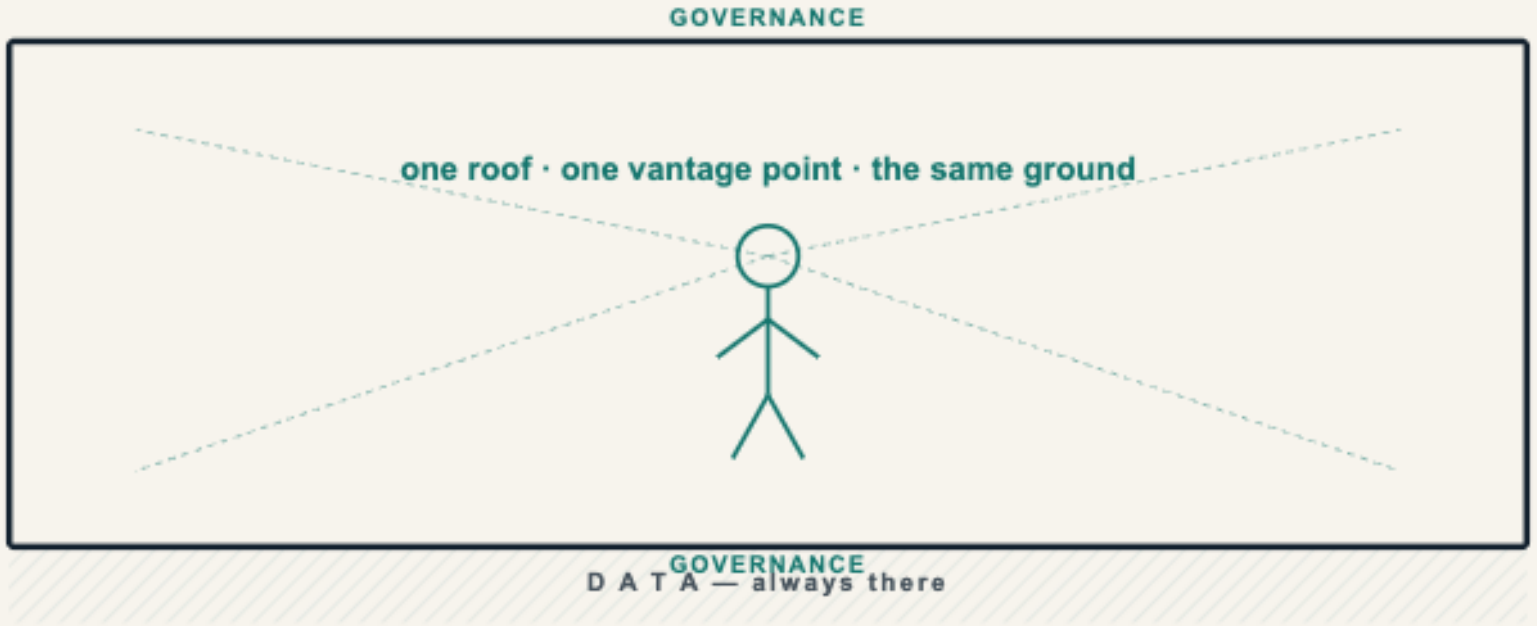
Ask in plain language, get an answer

For each era: what it solved, what it created, and what stayed the same.

Five rooms. One roof. The same ground.



Five rooms. One roof. The same ground.



Data trapped in the machine

Pre-cube era

```
MAIN                          IBM i Main Menu
                                                          System:
Select one of the following:

     1. User tasks
     2. Office tasks
     3. General system tasks
     4. Files, libraries, and folders
     5. Programming
     6. Communications
     7. Define or change the system
     8. Problem handling
     9. Display a menu
    10. Information Assistant options
    11. IBM i Access tasks

    90. Sign off

Selection or command
===>

F3=Exit    F4=Prompt    F9=Retrieve    F12=Cancel    F13=Information Assistant
F23=Set initial menu
(C) COPYRIGHT IBM CORP. 1980, 2009.
 ONLINE                                          M  20,7
```

Main Menu of IBM i 7.1, shown inside a TN5250 client. By Vt320, CC BY-SA 4.0, via Wikimedia Commons https://commons.wikimedia.org/wiki/File:Ibmi-main-menu.png .

What this represents

Data lived inside two different platforms, one shared problem. Mainframe VSAM files on MVS/z/OS — a file access method, not a database, holding billions of records no external system could easily read. Midrange AS/400 systems with DB2/400 built in — powerful, but proprietary. ***Getting an answer meant knowing the record layout***, the RPG job, the 5250 terminal command. The data was always there. The question had to speak the machine's language.

The discipline it required

Someone had to know the **record layout.** Someone had to own the **field definitions**. That was data architecture – before anyone called it that.

This is where most enterprise data lived in the 1980s and 90s. Green screen. Command line**. The data was there - it just didn't speak human yet**

The OLAP cube

Era 1 of 4



What it solved

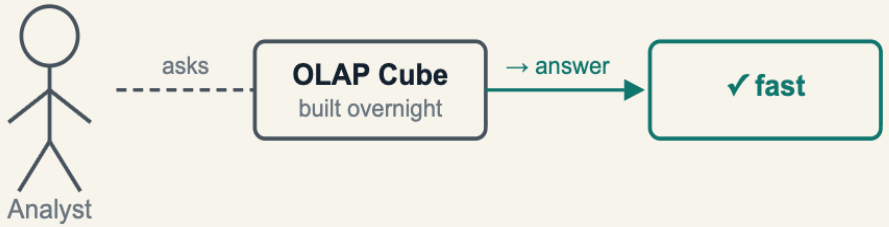
Fast, consistent answers to a known set of business questions.

What it created

Rigid models. A new question often meant a new cube and a nightly rebuild.

What stayed the same

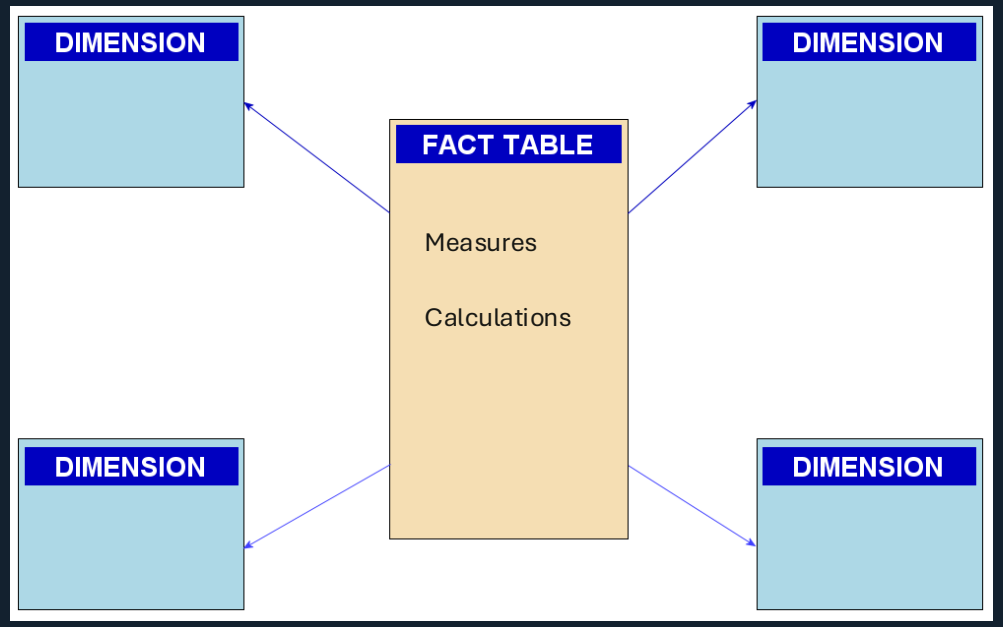
Someone had to define what "revenue" meant, and own that definition.





Story for this era: A retail merchandising analyst in 2003. She needs to know which product category drove last week's regional revenue drop. The cube has the answer -- if someone built that dimension last quarter. If they didn't, she waits three days for a nightly rebuild. The question was right. The machinery decided whether it was answerable.
The discipline here: someone had to define "revenue" before the cube could serve it. That ownership question never goes away - it just changes form in every era.

The pre-computed answer



"Star schema" by SqlPac, CC BY-SA 3.0, via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Star-schema.png#/media/File:Star-schema.png

What this represents

The star schema is the blueprint of the OLAP era. One fact table at the centre. Dimension tables around it — time, product, customer, geography. Every query was a join across this model. Fast, if you built the right dimensions in advance.

The discipline it required

Months of modelling before a single query ran. Someone had to define "revenue" before the cube could calculate it. The ownership question starts here.

Every query in this era had to travel through this model. If your question fit the model, you got an answer in milliseconds. If it didn't, you waited three days for a rebuild.

Buy a bigger box

The appliance peak

What it solved

Performance at scale. Purpose-built hardware ran queries faster than any general-purpose server could. The appliance was the answer to every complaint about speed.

What it created

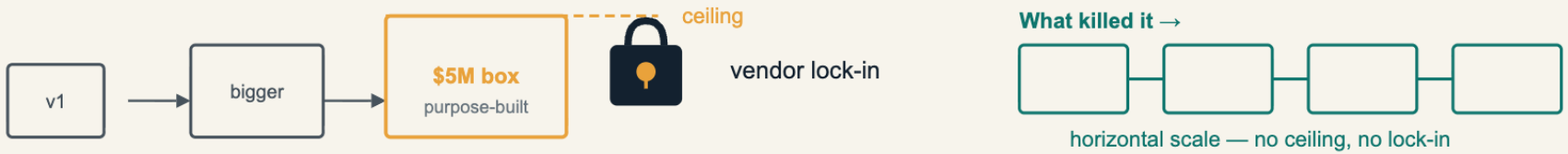
A $2–5M decision that locked you in for a decade. Compute and storage were coupled in fixed racks. You couldn't resize without redistributing data. You couldn't pay per use. You couldn't simply turn off what you weren't using. And a vendor owned every constraint.

What stayed the same

Someone still had to model the data, own the definitions, and justify the spend to the CFO.

Tools of the era

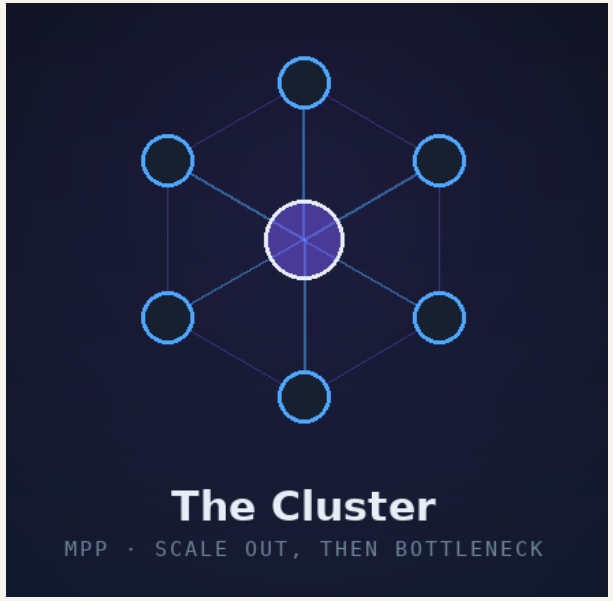
IBM PureData / Netezza | Oracle Exadata | Teradata | Sybase IQ | Vertica | Greenplum



This is the era were the most **capital expenditure conversation** happened. The vendor roadmap dependency, the $2–5M decision that locked you in for a decade (Computerworld, 2008; Monash Research, 2010). These were **scale-out MPP systems** — many nodes, parallel query — **not vertically scaled boxes**. The real constraint was that **compute and storage were coupled in fixed racks**. You couldn't resize without redistributing terabytes of data. You couldn't pay per use. **What killed it wasn't a better appliance. It was cloud compute and elastic storage** making the fixed-rack model's economics obsolete.

The MPP cluster

Era 2 of 4



What it solved

Scale. Loads and queries spread across many nodes instead of one big server.

What it created

Tuning became a specialty, and copies of the data multiplied across teams.

What stayed the same

Knowing where each column came from, and where it went next.



Story for this era: A bank running large appliance. Five business units each have their own definition of "active customer." The MPP cluster can query all five simultaneously at scale - it cannot tell you which definition is correct. MDM, Master Data Management, becomes the era's answer. Someone has to own the definition before the machine can serve it. Scale solved the performance problem and quietly made the lineage problem bigger, because the same data now lived in more places with more owners. This is the standardisation era — the rise of data governance as a practice, not just a tool. Target: 2 minutes.

The invisible engine room

What it solved

Moving data reliably from source to warehouse, on schedule, every night. ETL tools made complex transformations repeatable. Schedulers made sure they ran in the right order.

What it created

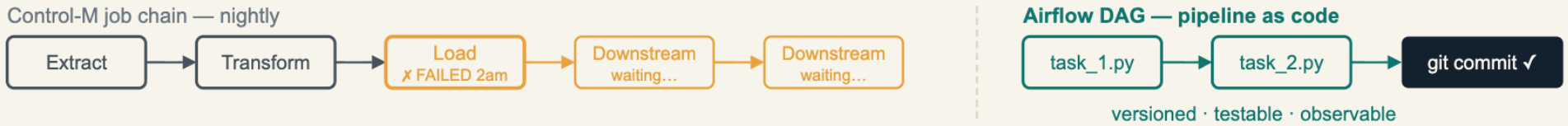
Thousands of brittle jobs nobody dared touch. War room screens monitoring job chains through the night. A single upstream failure could cascade through dozens of downstream loads by morning.

What stayed the same

Someone had to own each job, know its dependencies, and be on call when it broke at 2am.

Tools of the era

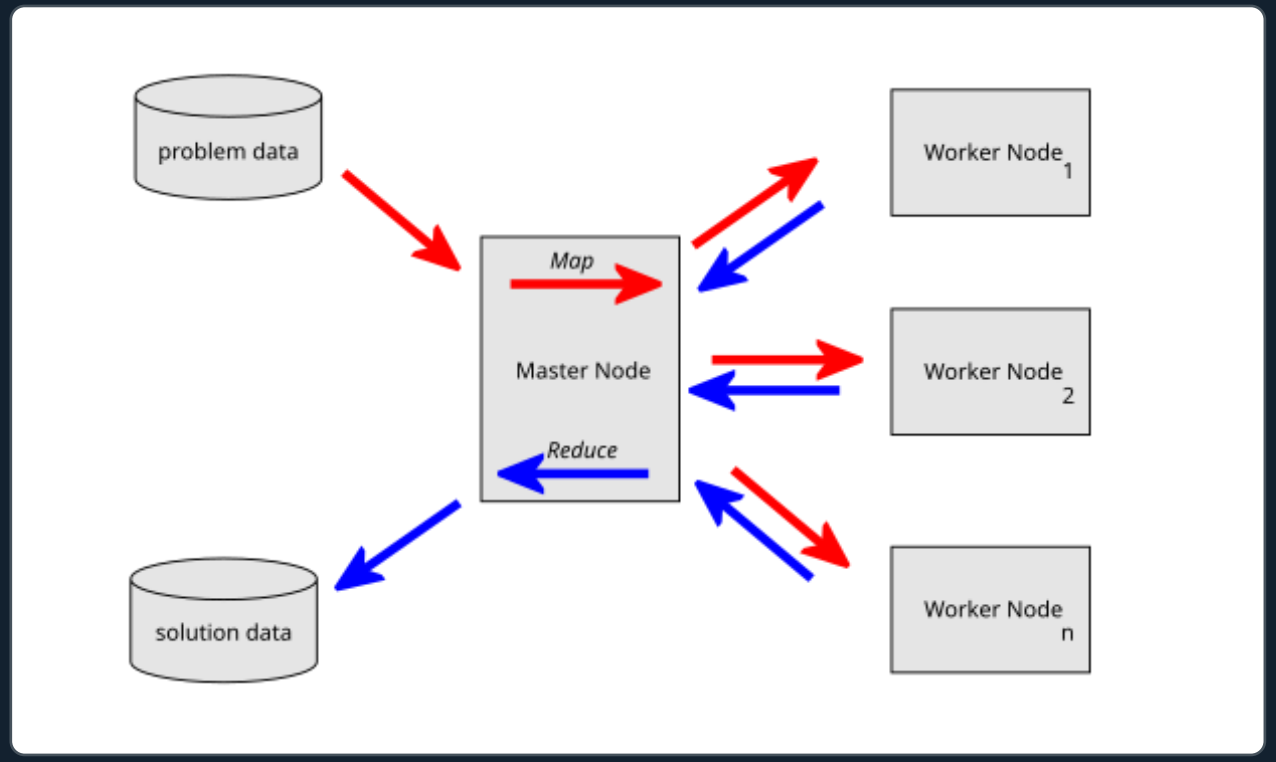
IBM DataStage | Informatica PowerCenter | SAP BODS | SSIS | **Ab Initio ↯** | Control-M | TWS / Tivoli | Autosys | Apache Airflow →



Ab Initio was the ghost in the machine. No marketing, no user conferences, no open documentation. If your company ran it, you knew. If it didn't, you'd probably never heard of it. “Some of the largest banks and telcos in the world ran their entire data operations on it." Schedulers: behind every ETL job was a scheduler: Control-M, Tivoli, Autosys – managing thousands of dependencies on a war room screen. Airflow didn't replace that complexity. It made it visible, versionable, and testable."

Spread the work. All the machines.

Hadoop era



"Mapreduce Overview" by Poposhka, CC BY-SA 3.0, via Wikimedia Commons / Wikipedia.
https://en.wikipedia.org/wiki/File:Mapreduce_Overview.svg

What this represents

MapReduce and MPP systems split one query across many nodes. No single machine needed to hold it all. Data could finally be as large as the business generated it – not as large as one server could fit.

The discipline it required

More nodes meant more copies, more lineage to track, more definitions to align. Scale solved the volume problem and quietly made the governance problem bigger.

One query, split across dozens of machines, reassembled at the end. The hardware constraint of a single server died here.
The governance constraint – “where did this data come from, who owns it” - got bigger.

Compute separates from storage

What it solved

Columnar storage made scanning hundreds of millions of rows in seconds possible without pre-calculating anything. Cloud-native designs — BigQuery and Snowflake — went further: they separated compute from storage entirely. Pay for what you use. Scale each independently. No fixed rack, no decade-long contract. Others followed.

What it created

A new kind of sprawl. Data in every cloud, every region, every team's private schema. Self-service BI put dashboards in everyone's hands — and five different numbers in every board deck. Gartner warned in 2015: "without appropriate governance, self-service capabilities introduce multiple versions of the truth."

What stayed the same

You still had to know the schema. You still had to model first. The machine was faster, not friendlier.

Tools of the era

SAP HANA | Google BigQuery | AWS Redshift | Azure Synapse | Snowflake | Tableau | Power BI | Looker



This is the era when the hardware constraint died and a new one was born. "BigQuery and Redshift didn't just make queries faster. They made the appliance's business model obsolete. You didn't buy compute anymore. You rented it by the second." Then the self-service BI ramped up: "for the first time, the analyst didn't need a SQL developer. She had a dashboard. The citizen-data-scientist, data science commoner era began here. **The cost:** three dashboards, three numbers, no agreed definition. Same problem as the Teradata MDM era - just wearing different clothes."

One platform. Warehouse and lake.

Lakehouse era

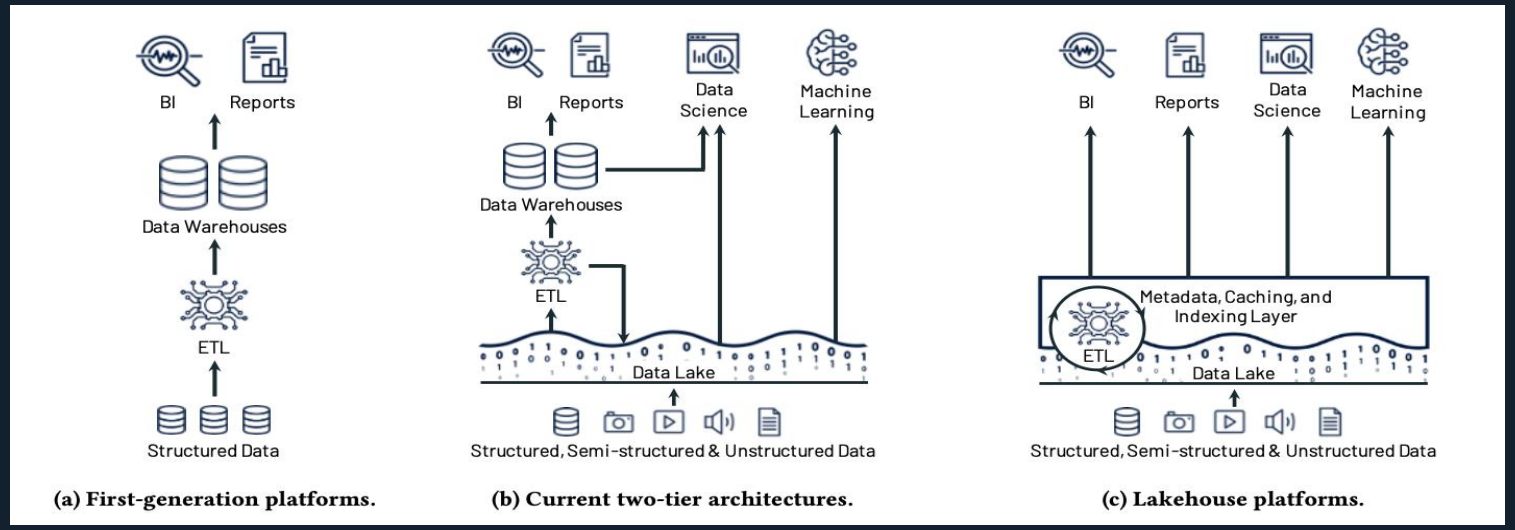


Fig. 1 from Armbrust, Ghodsi, Xin & Zaharia, "Lakehouse: A New Generation of Open Platforms," CIDR 2021. CC BY 3.0.
https://www.cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf

What this represents

The convergence of the data warehouse and the data lake into one open platform. One copy of data. BI, data science, and ML served from the same source. Bronze raw, Silver cleaned, Gold governed.

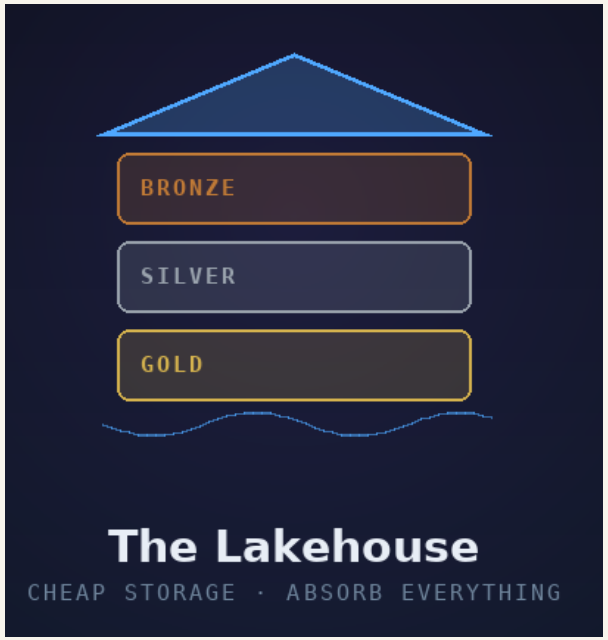
The discipline it required

Landing everything got easy. So did losing track of it. Each layer only helps if someone owns it. The medallion pattern is not a technology – it is a governance decision.

First the warehouse alone. Then the lake bolted on beside it, a two-tier mess. Then the two merged into one. That is the lakehouse. One floor. Two uses. Same data.

The lakehouse

Era 3 of 4



What it solved

One storage layer for BI, data science, and ML, organized in bronze, silver, and gold.

What it created

Landing everything got easy, and so did losing track of it. Layers only help if each has an owner.

What stayed the same

Quality checks and lineage between the layers, not only at the end.



Story for this era: 2013. A telco has clickstream data from 40 million mobile users. Customer-centric marketing means every campaign needs a different segment – no pre-built cube can anticipate all of them. The lake stores everything raw. A data scientist can finally ask questions nobody thought to pre-approve. The cost: she needs Spark, Python, and three weeks to get there. The lakehouse – bronze, silver, gold – was the answer to that cost: structure the raw data progressively so more people can use it without starting from scratch. The medallion pattern: the architecture gave everyone a place to put data, but not a reason to trust it. Layers only help if each one has an owner.

Zero-ETL: the pipeline you no longer see

Detour · Era

What it solved

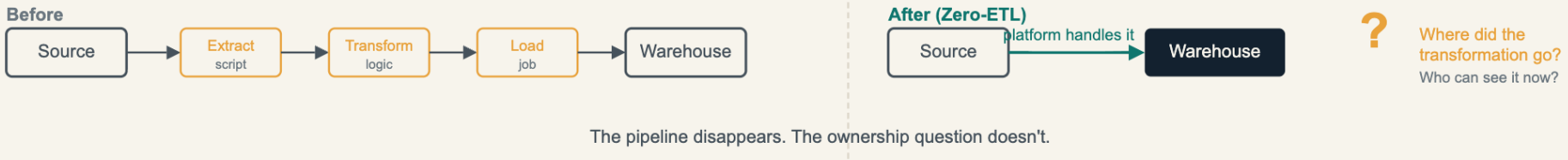
Data flows from source systems to analytics with no hand-built pipeline to write or babysit.

What it created

The transformation logic does not disappear. It moves downstream, where fewer people can see it.

What stayed the same

Someone still has to know what changed, what it means, and who owns it.

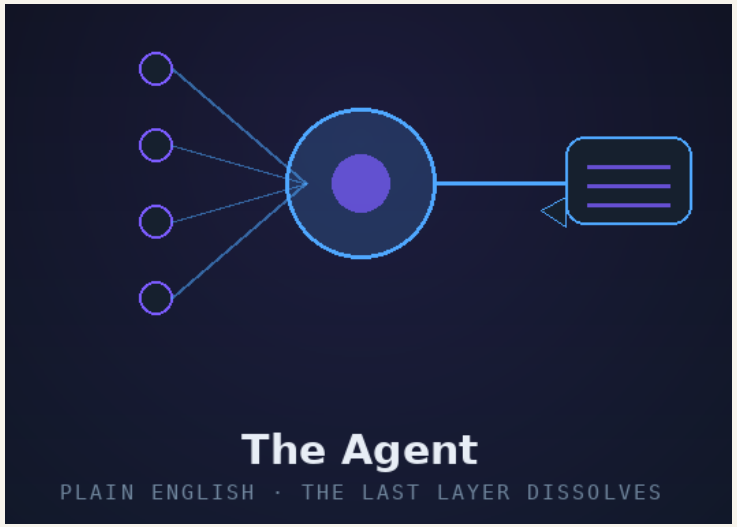


The pipeline disappears. The ownership question doesn't.

ETL: extract, transform, load – the three steps that move data from where it lives to where you can analyse it. Zero-ETL means the platform handles replication automatically; there is no pipeline to hand-build or babysit. That removes real toil. But here is the question: if the pipeline is gone, where did the transformation logic go, and who can see it now? It moved downstream – into the semantic layer, into the ontology, into the gold table definitions. This is the same lineage question as the cube era in 1998 and the MDM era in 2008. The ownership question doesn't disappear when the pipeline does. This is exactly why the AI agent works – the zero-ETL layer kept the data current, and the discipline in the layers above it made the answer trustworthy. Remove either, and the agent hallucinates confidently.

The agent

Era 4 of 4



What it solved

The machine starts meeting the user halfway – for the first time in 30 years, you don't need SQL. But only on top of governed, well-modeled data

What it created

Wrong answers arrive just as fast, and they sound confident.

Spider 2.0 · ICLR 2025. 21.3% of real enterprise NL2SQL tasks solved by the best AI agent. vs 91.2% on simple benchmarks. The gap is the data underneath.

What stayed the same

Lineage, shared definitions, and a way to prove the number is right.

Sales	Finance	Marketing	Product	Support
42,100	39,800	51,200	44,600	38,400

"active customers" — 5 teams, 5 answers

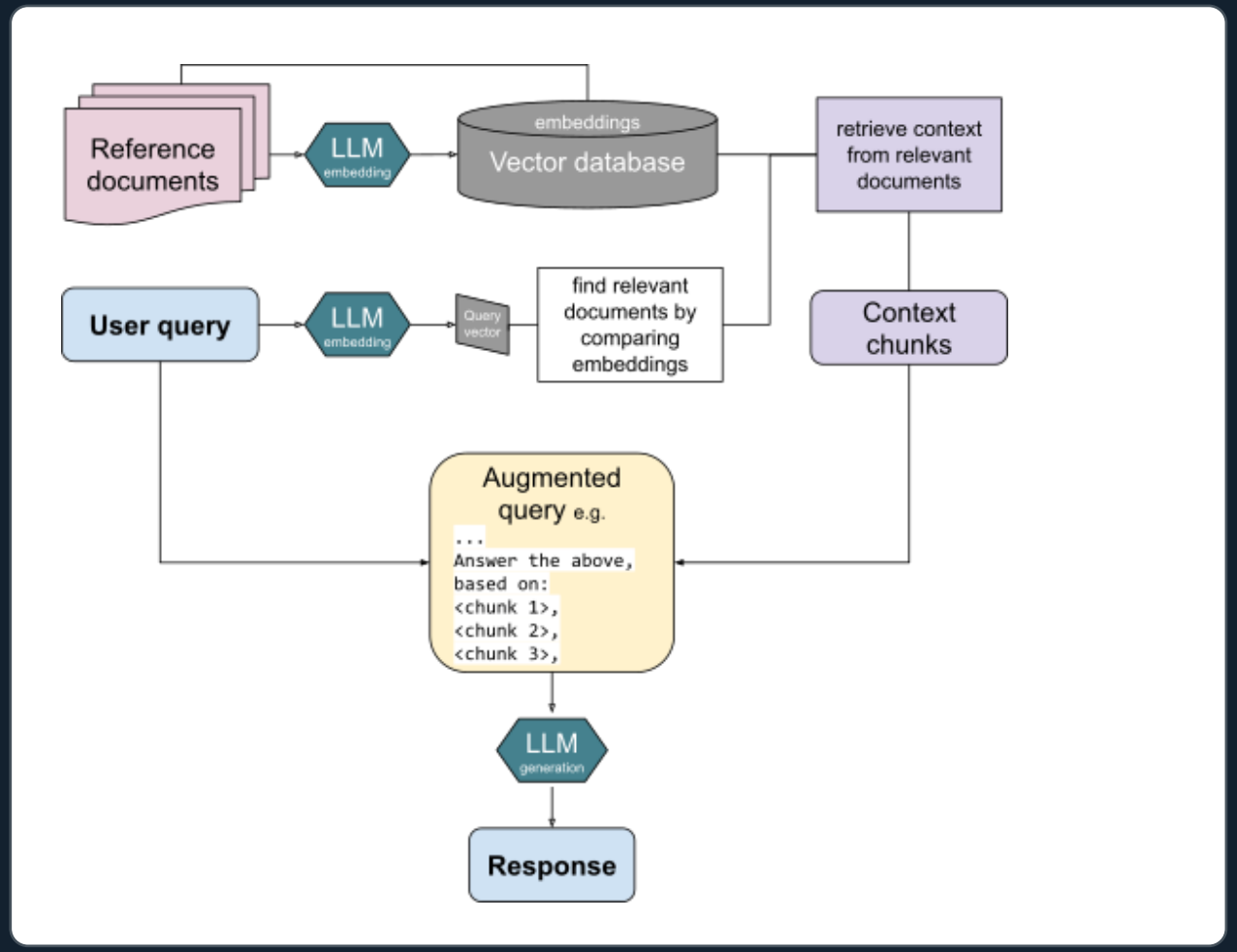
MDM → **One definition** 42,100

owned, governed, trusted

Story for this era: A nurse practitioner in a value-based care clinic. She has 200 patients with diabetes. She needs to know which ones haven't had an HbA1c test in the last 90 days and are at risk of a care gap penalty. In 2023, that is a 3–4 day email chain to the data team – routing, triage, SQL developer, SME review, data pull, delivery.
In 2025, she types the question in plain English and gets the answer in seconds. Why? Because someone built the ontology that maps "diabetes" to the correct ICD-10 codes. Because the pipeline discipline meant the data was clean and current. Because the governance layer made the answer trustworthy enough to act on. The agent didn't build any of that. It stood on top of it. This is the bridge to my thesis: an agent cannot invent the discipline – it needs the same lineage and definitions a cube needed in 2003, and it fails faster without them. Wrong answers now arrive in seconds, and they sound confident. Spider 2.0: "The best AI agent in a 2025 benchmark solved 21.3% of real enterprise SQL tasks. 21.3%. Not because the AI is bad – but because real enterprise data is messy, ungoverned, and poorly modeled. The discipline is the difference. It always was."

Plain English in. Trusted answer out.

Agent era



"RAG diagram" by Turtlecrown, CC BY-SA 4.0, via Wikimedia Commons.
https://commons.wikimedia.org/wiki/File:RAG_diagram.svg#/media/File:RAG_diagram.svg

What this represents

The agent retrieves relevant context from a knowledge store, grounds it with your data, and returns a trusted answer — no SQL required. The translation layer finally moves out of the human's hands.

The discipline it required

The agent stands on top of every prior era's discipline. Without clean data, governed lineage, and accurate ontologies underneath, the answer arrives in seconds — and it is wrong.

Ontologies: meaning a machine can read

The emerging layer

An ontology is an explicit model of your business concepts and how they relate, readable by people and by machines.



For an agent, it turns "active customers" into one governed definition instead of a guess. It still needs an owner.

example: a customer places an order, an order contains products. Simple, but now a system can follow those links instead of guessing. This is the discipline becoming machinery: the definition someone used to carry in their head, written down in a form an agent can use. Connecting back to the cube era, where someone had to define what "revenue" meant. An ontology without an owner goes stale like any other document.

The machinery changed. The discipline didn't.

Lineage. Governance. Knowing why a number is right.

Three things every era needed

The constant



Lineage

Where did this come from, and what touched it on the way?



Governance

Who owns the definition, and who is allowed to change it?

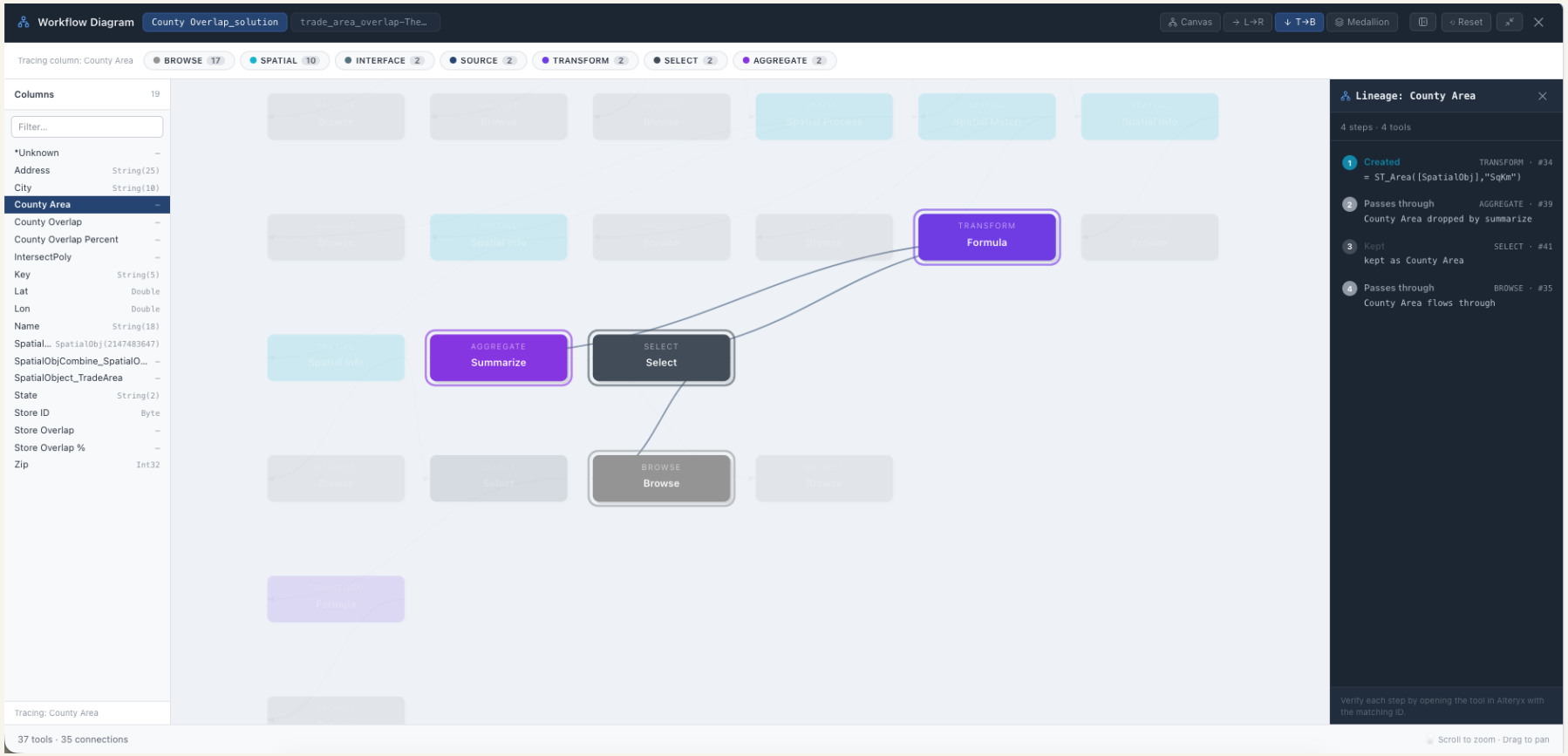


Proof

How would I know if this number were wrong?

A pipeline with 37 steps

Dozens of workflows in production, built by people who have moved on. Someone asks what happens to customer data between ingestion and the dashboard, and nobody is entirely sure.



37 tools

35 connections

22 steps

This is the story from one of [My article:] A workflow nobody could fully explain, and what it took to trace one column from origin to output. Lineage is a map people can read, not a document someone updated once.

The database feature that vanished

Story 2 · Proof

What happened

A migration tool silently removed a generated search column from the database.

Why nobody noticed

Vector search kept returning results, so nothing looked broken.

How it was found

Diagnostic SQL against the database catalog, not the application logs.

A system that still returns answers is not proof that it returns the right ones.

This comes from my own RAG build article. It is the cleanest example of a silent failure: the retrieval layer kept working, so the missing feature left no error. The takeaway is verification at the source of truth, not trust in the abstraction.

Three questions for any data system

Take this with you

1. Where did this come from?
2. Who is accountable for changing it?
3. How would I know if it were wrong?

Tools turn over. Fundamentals transfer.

For your career

What changes

- The platform
- The vendor
- The interface

What transfers

- Modeling and clear definitions
- Lineage and quality thinking
- Asking where a number came from

For the early-career people and career changers: Nobody can predict which platform will matter in five years, but the habit of asking where a number came from will keep paying off.

How do you know it's right?

Questions, stories, and disagreements welcome.

Prashanth Soma



https://www.linkedin.com/in/prashanthsoma

https://github.com/psoma01/data-architecture-talk

Primary sources

Seven citable sources behind the claims in this talk. Vendor-authored papers are marked ϟ

Inmon · 1992

Building the Data Warehouse

W.H. Inmon, Wiley. Original definition of a data warehouse as "subject-oriented, integrated, time-variant and non-volatile."

Chaudhuri & Dayal · ACM SIGMOD Record 1997

An Overview of Data Warehousing and OLAP Technology

dl.acm.org/doi/10.1145/248603.248616 — Standard academic survey of warehouse, star schema and OLAP architecture. Free PDF on sigmodrecord.org.

Kimball Group · 1996–present

Dimensional Modeling Techniques

kimballgroup.com — The Data Warehouse Toolkit (1996) introduced dimensional modeling. Techniques page is open access.

Dageville et al. · SIGMOD 2016 ϟ

The Snowflake Elastic Data Warehouse

info.snowflake.net/rs/252-RFO-227/images/Snowflake_SIGMOD.pdf — Documents compute/storage separation as a response to shared-nothing hardware constraints. Vendor-authored.

Armbrust et al. · CIDR 2021 ϟ

Lakehouse: A New Generation of Open Platforms

cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf — CC BY 3.0. Documents convergence of warehouse and lake. Databricks founders, vendor-authored.

Stonebraker & Pavlo · SIGMOD Record 2024

What Goes Around Comes Around… And Around

semanticscholar.org — "Major advancements have been primarily driven by changing hardware characteristics." The thesis in peer-reviewed form.

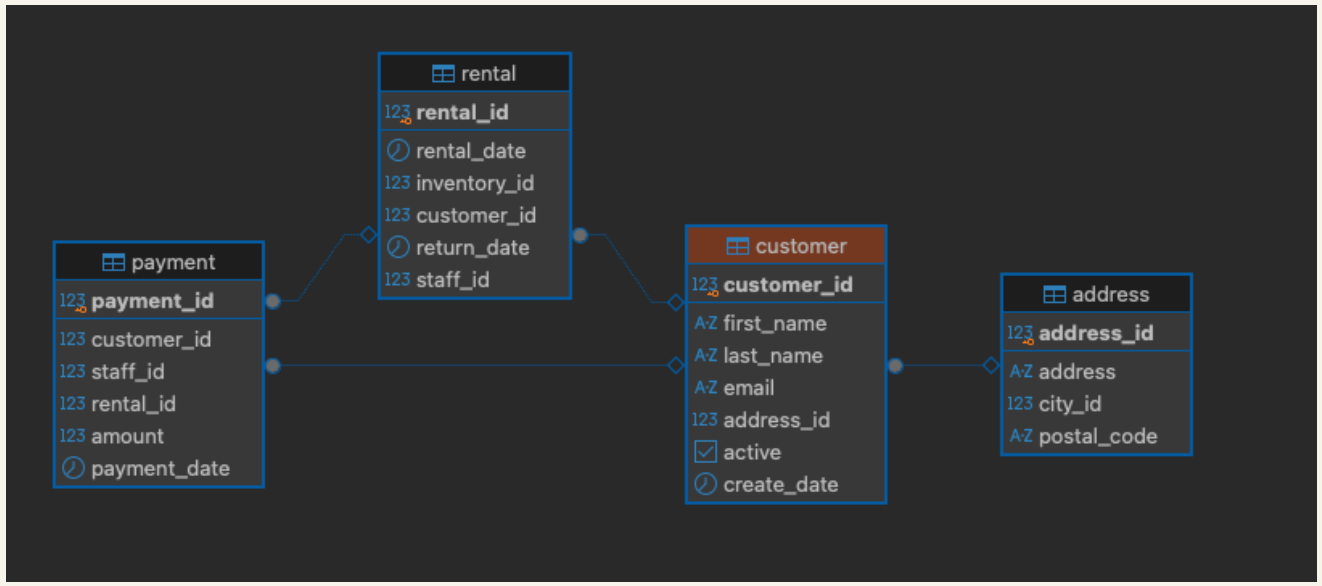
Lei et al. · ICLR 2025 (Oral)

Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows

arxiv.org/abs/2411.07763 — The best AI agent solved 21.3% of enterprise NL2SQL tasks vs 91.2% on simple benchmarks. The gap is governance and data quality. This is why the discipline still matters.

ϟ vendor-authored · all others are independent academic or analyst sources

PopstgreSQL - Agent




```
[soma@Macmini ~> psql pagila_demo -c "SELECT COUNT(*) FROM customer;"
 count
-------
    10
(1 row)

[soma@Macmini ~> psql pagila_demo -c "SELECT * FROM customer;"
 customer_id | first_name | last_name |       email         | address_id | active |        create_date
-------------+------------+-----------+---------------------+------------+--------+----------------------------
           1 | Alice      | Martin    | alice@example.com   |          1 | t      | 2026-10-20 21:11:37.365888
           2 | Bob        | Singh     | bob@example.com     |          2 | t      | 2026-10-20 21:11:37.365888
           3 | Carol      | Zhang     | carol@example.com   |          3 | t      | 2026-10-20 21:11:37.365888
           4 | David      | Okafor    | david@example.com   |          4 | t      | 2026-10-20 21:11:37.365888
           5 | Elena      | Kovacs    | elena@example.com   |          5 | t      | 2026-10-20 21:11:37.365888
           6 | Frank      | Diallo    | frank@example.com   |          1 | f      | 2026-10-20 21:11:37.365888
           7 | Grace      | Tanaka    | grace@example.com   |          2 | t      | 2026-10-20 21:11:37.365888
           8 | Hassan     | Ali       | hassan@example.com  |          3 | t      | 2026-10-20 21:11:37.365888
           9 | Ingrid     | Svensson  | ingrid@example.com  |          4 | t      | 2026-10-20 21:11:37.365888
          10 | James      | Patel     | james@example.com   |          5 | t      | 2026-10-20 21:11:37.365888
(10 rows)
```

...-talk/nl2sql-stub — vim agent.py ~/d/d/nl2sql-stub — vim agent.py

```
"""
agent.py — The three-step agent loop
====================================
Orchestrates SQL generation → judge validation → execution.
This is the file to show during the demo — the full chain in 40 lines.
"""

from dataclasses import dataclass, field
from typing import Optional

from llm import generate_sql, judge_validate
from db import execute_select


@dataclass
class AgentResult:
    question:  str
    sql:       str       = ""
    rows:      list[dict] = field(default_factory=list)
    validated: bool      = False
    error:     Optional[str] = None


def run(question: str, verbose: bool = True) -> AgentResult:
    """
    Run the full NL2SQL agent loop for a plain-English question.

      Step 1 — generate_sql()    : LLM writes a SELECT from the question
      Step 2 — judge_validate()  : second LLM call checks it before execution
      Step 3 — execute_select()  : runs against Postgres only if Step 2 passes

    The agent never executes SQL that fails validation.
    That is the discipline. Same discipline as every prior era.
    Different machinery.
    """
    result = AgentResult(question=question)

    if verbose:
"agent.py" 95L, 3441B
```

Local AI Agent

```
(soma@Mac ~/d/data-architecture-talk (main)> uv run python nl2sql-stub/demo.py "show me all films rated PG"       (base) ]
warning: The `tool.uv.dev-dependencies` field (used in `pyproject.toml`) is deprecated and will be removed in a future r
elease; use `dependency-groups.dev` instead

============================================================
  NL2SQL Agent — Local Demo
  Plain English → SQL generation → Judge validation → Postgres
============================================================

  Pre-flight checks:
  ------------------------------
  ✓ Ollama: phi4-mini:latest ready
  ✓ Postgres: pagila_demo ready (10 customers)


============================================================
  Question: show me all films rated PG
============================================================

  [1/3] Generating SQL...
  → SELECT title, description, release_year, rental_rate, length, rating
FROM film
WHERE rating = 'PG';

  [2/3] Validating with judge model...
  → ✓ PASS: PASS: SELECT statement is correct, and the WHERE clause is used to filter PG-rated films.

  [3/3] Executing against Postgres...

  ✓ 2 row(s) returned

+----------------+---------------------------------+--------------+--------------+--------+--------+
| title          | description                     | release_year | rental_rate  | length | rating |
+----------------+---------------------------------+--------------+--------------+--------+--------+
| ACADEMY DINOSAUR | Epic drama of a teacher and student | 2006 | 0.99 | 86 | PG |
| AGENT TRUMAN | Thriller on espionage | 2006 | 2.99 | 169 | PG |
+----------------+---------------------------------+--------------+--------------+--------+--------+
```

```
...e ~/d/data-architecture-talk — vim nl2sql-stub/schema_context.txt        ~ — ollama serve ~ — llama-server • ollama serve

-- NL2SQL Schema Context
-- This file is the data dictionary the agent reads before generating SQL.

Base tables:
  customer    (customer_id, first_name, last_name, email, address_id, active, create_date)
  film        (film_id, title, description, release_year, rental_rate, length, rating)
  category    (category_id, name)
  film_category (film_id, category_id)
  inventory   (inventory_id, film_id, store_id)
  rental      (rental_id, rental_date, inventory_id, customer_id, return_date, staff_id)
  payment     (payment_id, customer_id, staff_id, rental_id, amount, payment_date)
  city        (city_id, city, country)
  address     (address_id, address, city_id, postal_code)

Views — use these instead of joining base tables manually:

  active_rentals columns:
    rental_id, customer_name, film_title, category,
    rental_date, return_date, status
  Rules:
    - return_date IS NULL means the film has not been returned yet
    - status is either 'Returned' or 'Not returned'
    - Do NOT join other tables into this view — all data is already included
    - Do NOT use COUNT(*) or GROUP BY with this view — just SELECT and WHERE

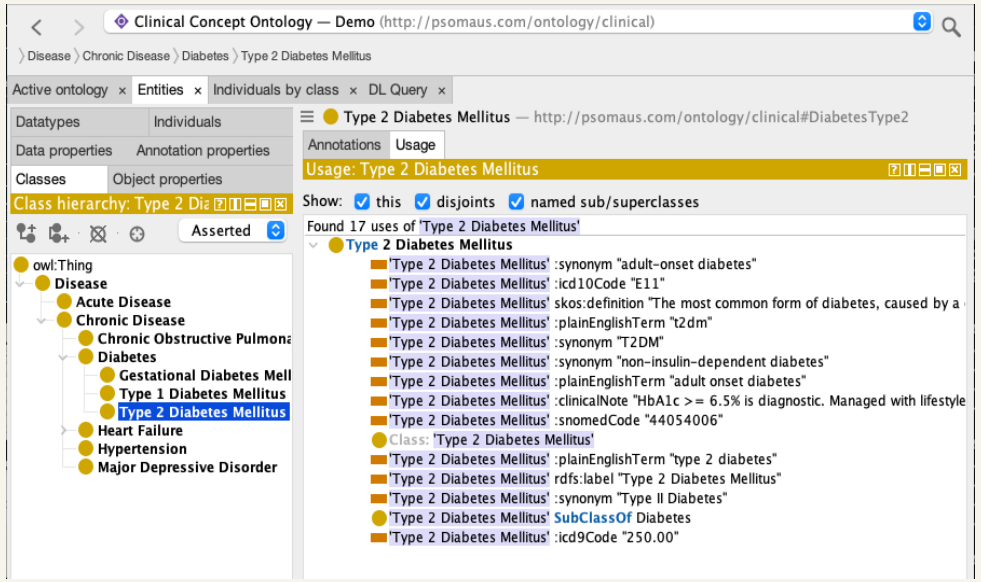
  customer_spend columns:
    customer_id, customer_name, email, city,
    total_rentals, total_spent, last_payment
  Rules:
    - This view is already aggregated — do NOT use COUNT(*), SUM(), or GROUP BY
    - total_rentals (INTEGER) — number of rentals per customer
    - total_spent (NUMERIC) — total payment amount per customer
    - Just SELECT the columns you need, ORDER BY, and LIMIT

SQL rules:
  - Use column names exactly as listed above — no guessing
  - Prefer views over manual joins
  - Only write SELECT statements — never INSERT, UPDATE, or DELETE
"nl2sql-stub/schema_context.txt" [noeol] 39L, 1771B
```

Ontology

Concepts in the ontology

```
Thing
└── Disease
    ├── ChronicDisease
    │   ├── Diabetes
    │   │   ├── DiabetesType2   (E11)
    │   │   ├── DiabetesType1   (E10)
    │   │   └── GestationalDiabetes (O24.4)
    │   ├── HeartFailure        (I50)
    │   │   └── HeartFailureReducedEF (I50.2)
    │   ├── Hypertension        (I10)
    │   ├── COPD                (J44)
    │   └── Depression          (F32)
    └── AcuteDisease
```




```
[soma@Mac ~/d/data-architecture-talk (main)> uv run python ontology-demo/ontology_lookup.py "high blood pressure"
warning: The `tool.uv.dev-dependencies` field (used in `pyproject.toml`) is deprecated and will be removed in a future release; use `dependency-groups.dev` instead

  Ontology loaded: 12 concepts

  Resolving: "high blood pressure"
  ──────────────────────────────────────────────────────────

  Found 1 match(es):


  ✓ Hypertension  (confidence: 90%)
    Parent: ChronicDisease
    ICD-10:  I10
    ICD-9:   401
    SNOMED:  38341003
    Synonyms: HTN, high blood pressure, arterial hypertension
    Definition: A long-term medical condition in which the blood pressure in the arteries is persistently elevated.
    Clinical note: Defined as systolic BP >= 130 mmHg or diastolic BP >= 80 mmHg (ACC/AHA 2017 guidelines).

  ──────────────────────────────────────────────────────────

  Agent payload (JSON):
  {
      "query": "high blood pressure",
      "resolved_label": "Hypertension",
      "icd10_codes": [
          "I10"
      ],
      "icd9_codes": [
          "401"
      ],
      "snomed_codes": [
          "38341003"
      ],
      "synonyms": [
          "HTN",
          "high blood pressure",
          "arterial hypertension"
      ],
      "confidence": 90,
      "parent_concept": "ChronicDisease"
  }

soma@Mac ~/d/data-architecture-talk (main)>                                    (base)
```

MCP - docstrings

```python
# ── MCP Tool Definition ──────────────────────────────────────────

def resolve_clinical_term(plain_english_term: str) -> dict:
    """
    Resolve a plain-English clinical term to structured medical codes.

    Use this tool BEFORE generating any SQL query that involves a
    medical condition, disease, diagnosis, or symptom. This ensures
    the query uses the correct ICD-10 codes rather than guessing from
    free-text column values.

    When to call this tool:
    - The user's question mentions a disease, condition, or symptom
      (e.g. "patients with diabetes", "members diagnosed with CHF",
      "beneficiaries with high blood pressure")
    - You need to filter a clinical database by diagnosis or condition
    - You are unsure which ICD-10 code family applies to a term

    Args:
        plain_english_term (str):
            The condition or disease as the user stated it — do not
            translate or normalise before passing. Examples:
            "diabetes", "high blood pressure", "heart failure", "CHF",
            "Type 2 diabetes", "COPD", "depression"

    Returns:
        dict with keys:
            query           (str)  — the original term you passed in
            resolved_label  (str)  — preferred clinical label
            icd10_codes     (list) — ICD-10 code(s) to use in SQL WHERE clause
            icd9_codes      (list) — ICD-9 codes if available
            snomed_codes    (list) — SNOMED CT codes if available
            synonyms        (list) — accepted alternative names
            confidence      (int)  — match confidence 0–100
            parent_concept  (str)  — where this fits in the hierarchy
            not_found       (bool) — True if no match; ask the user to clarify

    Example:
        Input:  "diabetes"
        Output: {
            "query": "diabetes",
            "resolved_label": "Diabetes",
            "icd10_codes": ["E08-E13"],
            "confidence": 100,
            "parent_concept": "ChronicDisease",
            ...
        }
```

data-architecture-talk > ontology-demo > mcp_ontology_tool.py > ...

```python
def resolve_clinical_term(plain_english_term: str) -> dict:
        Output: {
            "query": "high blood pressure",
            "resolved_label": "Hypertension",
            "icd10_codes": ["I10"],
            "snomed_codes": ["38341003"],
            "confidence": 90,
            ...
        }

    Important:
        If not_found is True, do NOT guess an ICD-10 code. Instead,
        ask the user to clarify the condition before generating SQL.
        A wrong code silently returns wrong patients — there is no
        error, only a bad answer that sounds correct.
    """
    if not _CONCEPTS:
        return {
            "query":          plain_english_term,
            "error":          "Ontology file not loaded. Check that clin
            "not_found":      True,
        }

    matches = resolve_term(plain_english_term, _CONCEPTS)

    if not matches:
        return {
            "query":          plain_english_term,
            "resolved_label": None,
            "icd10_codes":    [],
            "icd9_codes":     [],
            "snomed_codes":   [],
            "synonyms":       [],
            "confidence":     0,
            "parent_concept": None,
            "not_found":      True,
            "suggestion":     "Term not found in ontology. Ask the user
        }

    top = matches[0]
    return {
        "query":          plain_english_term,
        "resolved_label": top["label"],
        "icd10_codes":    top["icd10"],
        "icd9_codes":     top["icd9"],
        "snomed_codes":   top["snomed"],
        "synonyms":       top["synonyms"],
        "confidence":     top["_score"],
        "parent_concept": top["parent"],
        "not_found":      False,
```

```
The agent sees this tool description at runtime:
──────────────────────────────────────────────────────────────────
Tool name:  resolve_clinical_term
Description:
Resolve a plain-English clinical term to ICD-10 and SNOMED codes
turns structured codes, synonyms, and confidence score. If not_fou
──────────────────────────────────────────────────────────────────


The agent reasons: 'The user asked about patients with
diabetes. I must call resolve_clinical_term BEFORE
generating SQL, or I will guess the wrong codes.'


Agent calls: resolve_clinical_term('diabetes')
→ resolved_label: Diabetes
→ icd10_codes:    ['E08-E13']
→ confidence:     100%
→ not_found:      False

Agent calls: resolve_clinical_term('high blood pressure')
→ resolved_label: Hypertension
→ icd10_codes:    ['I10']
→ confidence:     90%
→ not_found:      False

Agent calls: resolve_clinical_term('chf')
→ resolved_label: Heart Failure
→ icd10_codes:    ['I50']
→ confidence:     90%
→ not_found:      False

Now the agent generates SQL using I10, I50, E08-E13
— not a free-text LIKE '%diabetes%' guess.

This is the discipline the ontology provides.
The same discipline the star schema required in 2003.
The machinery changed. The discipline didn't.
```